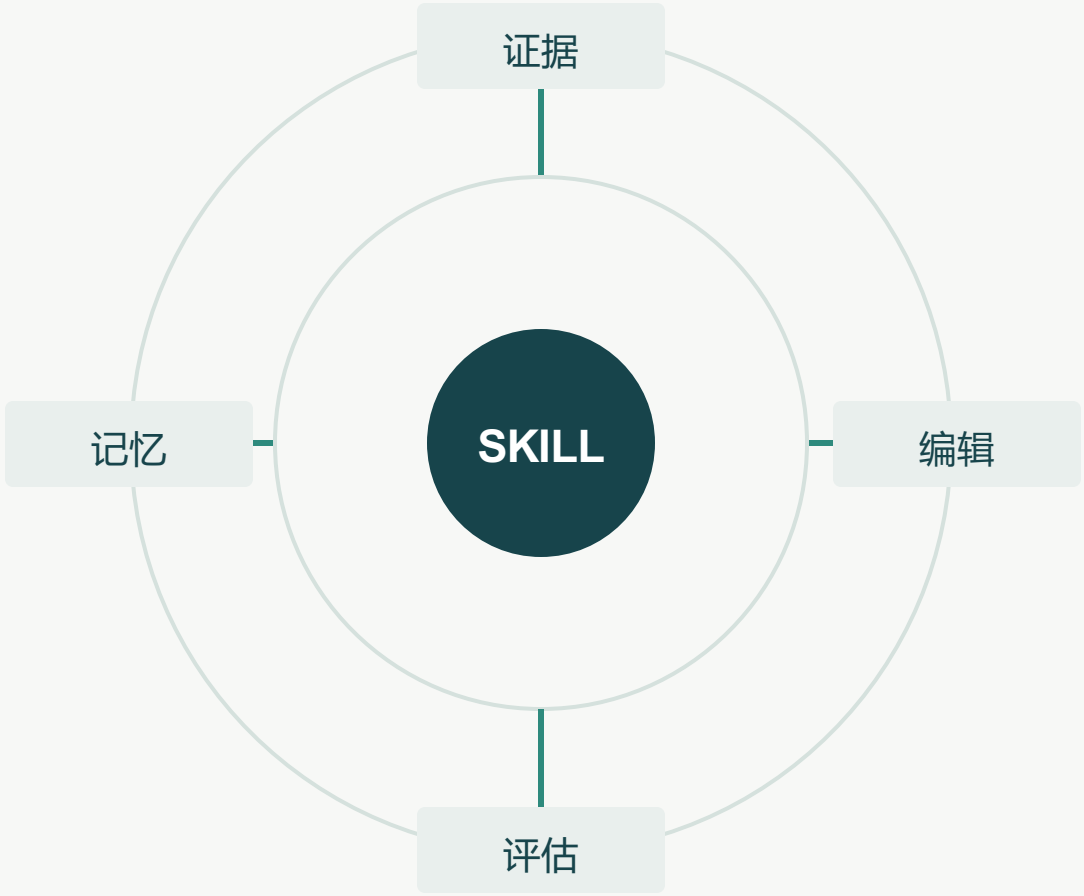
SKILL 自进化

Governed Skill Evolution

从 benchmark 设计到持续 Memory

基准实验 · 持续 Memory · SkillAdam

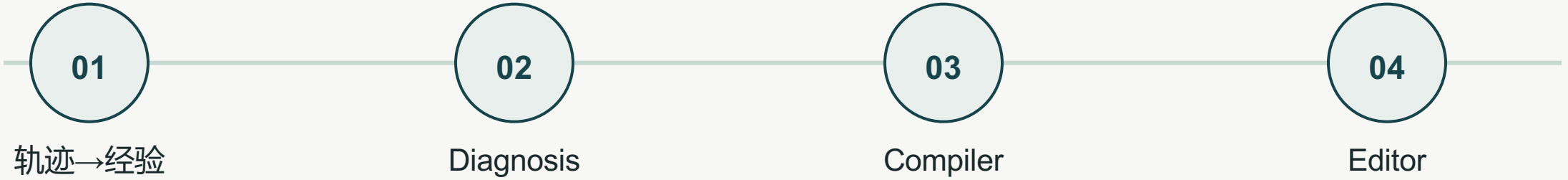


从任务轨迹到可执行的 Skill 修改

Benchmark：81 个任务



每轮自进化

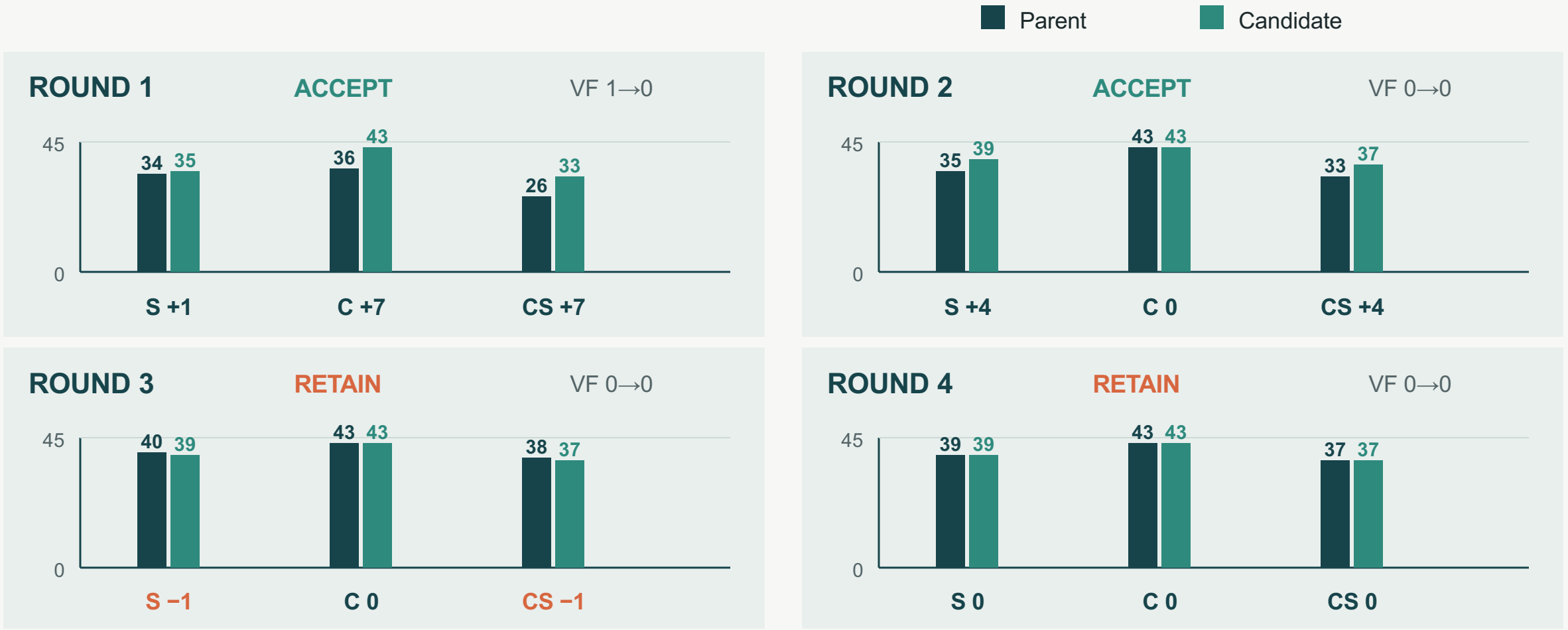


设计流程

先将 Evolution 轨迹转为 Agent 可见经验，再按共同决策问题分组。Diagnosis 从组内轨迹中找出有证据、可执行的局部规律，并判断当前 Skill 是否缺少规则、需要修订规则或应删除规则。

Compiler 固定映射 ADD / REPLACE / DELETE；Editor 写新增与替换文本，DELETE 由程序执行。

四轮 Monitor：前两轮接受，后两轮保留



最终保留第 2 轮：相对初始 S +5 / C +7 / CS +11 / VF −1；多数问题在一两轮内修复， 说明benchmark太简单，收益很快到头。

45 个 Monitor；S 成功 / C 合规 / CS 成功合规 / VF 失败违规。

一次整体决定，不应抹去单项修改的历史

一份 Candidate 可能包含多个 edit

Gate 只给整体 ACCEPT / RETAIN

01

保留被拒绝方案中的价值

整份 Candidate 被 RETAIN，
合理的局部 edit 仍可在下轮
重新核验和利用。

02

继续修订已采用的规则

即使整体 ACCEPT，已进入
Parent Skill 的 edit 仍要接受
新证据和反例的检验。

03

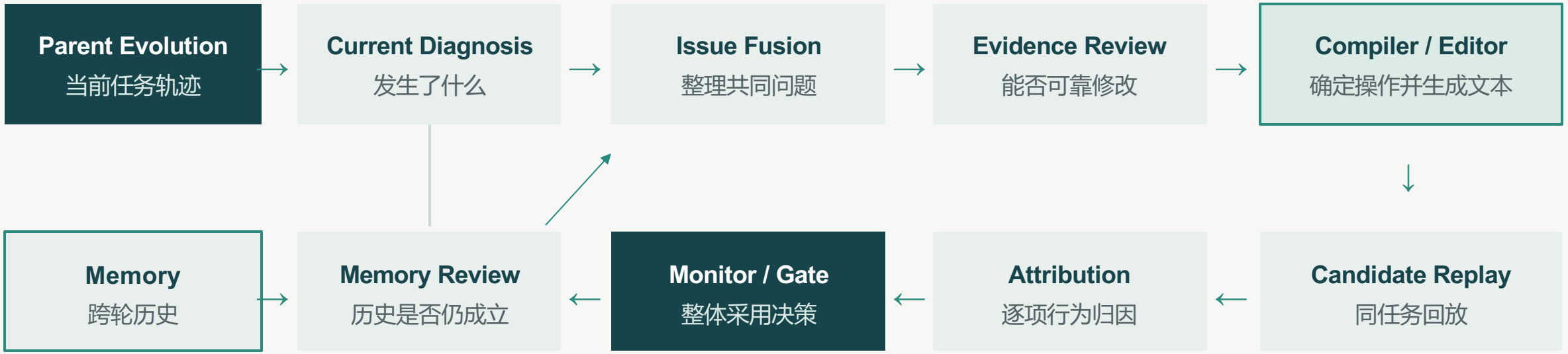
记住失败尝试的限制

记录方案失败在哪些条件下，
再遇到同类问题时避免机械重复。

持续 Memory = 问题 → 尝试 → 行为与结果 → 部署状态 → 后续修订

每项 edit 独立保存证据和限制，整体 Gate 结果不能替代局部历史。

当前证据与历史经验共同决定下一次修改



Gate 决定整份 Candidate；Memory 保存每项 edit 的证据、限制与部署状态。

Gate 后更新 Memory，下轮重新 Review；接受时 Candidate 成为 Parent，否则保留原 Parent。

引入Memory后的具体流程

01 Current Diagnosis

当前诊断

只看当前轨迹和约束

问题：失败、违规或误用

有效行为：已验证可行

反证：限制已有判断

不确定：证据尚不足

输出：有原始证据的发现

02 Issue Fusion

历史记忆复查

用生成的新证据重审历史修改

支持：新证据印证旧判断

反驳：新证据否定旧判断

混合：支持与反例并存

未覆盖：缺少相关新证据

保留已知反例和限制

03 Evidence Review

问题融合

当前发现 + 历史经验

合并同类问题

提出：值得继续核验

暂不修改：证据不足

值得继续核验?

04 Evidence Review

证据核验

逐问题核验证据

反例 / 适用范围

工具能力 / 历史限制

核实通过：可修改
暂不修改：证据不足或无需改

能否形成可靠修改?

整体 Gate 负责采用，逐项 Attribution 负责学习

确定性编辑 Compiler / Editor

缺少规则→新增

现有规则需修订→替换

现有规则应删除→删除

Editor 写新增与替换文本；
DELETE 由程序执行。

同任务配对回放 Candidate Replay Attribution

Parent <-> **Candidate**

相同的 36 个 Evolution 任务

Attribution 记录关联改善、退步、
无明显效果和无法归因的变化。

关联观察 ≠ 单项 edit 的独立因果效果

整体决策 + 局部回写 Monitor / Gate

Monitor / Gate

45 个独立任务决定 ACCEPT / RETAIN

Memory Update

保存每项 edit 的证据与部署状态

REVISES：继续修订已有 edit
RETRIES：重新尝试此前没有成功的修改方向
REVERTS：撤销此前的修改

两轮初步观察 · Monitor

第 1 轮 接受 **S 29→35** **C 37→41** **CS 24→31**

Memory 保存六项 edit 的局部证据

第 2 轮 拒绝 **S 35→31** **C 37→41** **CS 27→28**

生成 1 个 REPLACE、2 个 ADD

Memory 合理，主要问题是：知道第一轮哪里有风险之后，提出了一个过于保守的修复方案

Memory 已可记录历史，但其效果仍待证明

当前问题

01 有效性尚未验证

是否修订退步规则、保留有据可循的方向、避免重复失败

02 调用和上下文的成本高

逐 edit Review、逐任务 Attribution 随历史积累增加 token。

03 局部归因与整体泛化错位

Memory 用 Evolution 证据更新，Gate 看独立 Monitor。

下一步方向

A Memory设计结束后，
实现具体 Skill / 抽象 Skill 分层

B 把这版Skill自进化的方法
进行场景迁移与不同 benchmark

Memory 机制与原本双维度 Skill 演化的关系感觉不是很大。当前持续 Memory 更偏向“如何利用跨轮经验”。
原本的核心是如何同时利用 Success 和 Compliance 两个维度学习 Skill，是否可以设计到目前即可，不需要太细或者深入。

当前结论：两轮结果说明机制可运行，不能证明持续 Memory 带来增益。

SkillAdam：利用历史状态调节文本 Skill 的更新

初始化　从一部分任务的执行轨迹与评价反馈生成初始 Skill

每轮随机采样 mini-batch

Parent
轨迹 / 得分 / 失败反馈

→ **模型生成修改**
轨迹 + Memory + 幅度

→ **Candidate**
同一 mini-batch 回放

→ **逐任务配对**
更新历史与修改幅度

→ **Gate**
决定整体是否采用

借鉴 Adam：Memory 和动态修改幅度

Memory

记录过去的问题、修改尝试、效果和解决状态，
避免重复处理或破坏已解决的问题。

动态修改幅度

跨任务一致改善 → 适当扩大修改范围；
好坏波动明显 → 缩小更新幅度。